

Explain median uniqueness array

Absolutely. This problem looks scary because of the wording "uniqueness array", but the actual idea is a beautiful combination of:

1. Sliding Window
2. Counting distinct elements
3. Binary Search on the answer

Let's first understand the problem very clearly, without jumping into code.

1. What is the "uniqueness array"?

Given:

```
nums = [1, 2, 3]
```

Consider every possible subarray.

Length 1

```
[1] → 1 distinct  
[2] → 1 distinct  
[3] → 1 distinct
```

Length 2

```
[1,2] → 2 distinct  
[2,3] → 2 distinct
```

Length 3

```
[1,2,3] → 3 distinct
```

So the distinct counts are:

```
1, 1, 1, 2, 2, 3
```

Sort them:

```
[1, 1, 1, 2, 2, 3]
```

That's the uniqueness array.

2. What do we actually need?

We don't need to construct this array.

We only need its median.

For:

```
[1,1,1,2,2,3]
```

There are 6 elements.

The problem says:

```
If there are two choices for median, take the smaller one.
```

So the median is the element at position:

```
(n - 1) / 2
```

For 6 elements:

```
(6 - 1) / 2 = 2
```

0-indexed:

```
[1, 1, 1, 2, 2, 3]  
      ↑
```

Answer:

```
1
```

3. The important observation

Suppose we choose some number k .

We can ask:

How many subarrays have at most k distinct elements?

For example:

`nums = [1, 2, 3]`

Take:

`k = 2`

Subarrays having at most 2 distinct elements:

<code>[1]</code>	<code>→ 1</code>
<code>[2]</code>	<code>→ 1</code>
<code>[3]</code>	<code>→ 1</code>
<code>[1, 2]</code>	<code>→ 2</code>
<code>[2, 3]</code>	<code>→ 2</code>

But:

`[1, 2, 3] → 3`

doesn't count.

Therefore:

`count = 5`

4. Why is this useful?

Our uniqueness array is:

`[1, 1, 1, 2, 2, 3]`

Ask:

How many values are ≤ 2 ?

Answer:

`5`

Ask:

How many values are ≤ 1 ?

Answer:

`3`

Ask:

How many values are ≤ 0 ?

Answer:

`0`

This gives us a way to locate the median without creating the uniqueness array.

5. What position do we need?

There are:

`n = nums.length`

The number of all subarrays is:

$$n * (n + 1) / 2$$

Let's call this:

```
total
```

For:

```
nums = [1,2,3]
```

we have:

$$\begin{aligned} \text{total} &= 3 * 4 / 2 \\ &= 6 \end{aligned}$$

The median position is:

$$(total - 1) / 2$$

But because we're going to count values $\leq k$, it's convenient to use:

$$\text{target} = (total + 1) / 2$$

For 6:

$$\begin{aligned} \text{target} &= (6 + 1) / 2 \\ &= 3 \end{aligned}$$

So we want the **smallest** `k` such that at least 3 subarrays have $\leq k$ distinct elements.

6. Let's see it manually

Uniqueness array:

```
[1,1,1,2,2,3]
```

We need the 3rd element.

Try:

```
k = 1
```

Number of elements `<= 1`:

```
[1,1,1]
```

Count:

```
3
```

That's enough.

Therefore:

```
answer = 1
```

7. Now the big question

How do we efficiently calculate:

Number of subarrays having at most `k` distinct elements?

This is where **Sliding Window** comes in.

8. Sliding Window

Take:

```
nums = [1,2,1,3]
```

Suppose:

```
k = 2
```

We want subarrays having at most 2 distinct elements.

Start:

```
left = 0
right = 0
```

Window:

```
[1]
```

Distinct:

```
{1}
```

Valid.

Now:

```
[1,2]
```

Distinct:

```
{1,2}
```

Valid.

Now:

```
[1,2,1]
```

Distinct:

```
{1,2}
```

Still valid.

Now add 3:

```
[1,2,1,3]
```

Distinct:

```
{1,2,3}
```

That's 3 distinct, but we only allow 2.

So move left.

```
[2,1,3]
```

Still 3 distinct.

Move left again:

```
[1,3]
```

Now:

```
{1,3}
```

2 distinct → valid.

9. The most important sliding-window trick

Suppose our current valid window is:

```
[left ..... right]
```

and it has at most `k` distinct elements.

Then **every subarray ending at `right` and starting from `left` or later is also valid.**

For example:

```
nums = [1,2,1]
```

Window:

```
[1,2,1]
```

There are 2 distinct elements.

So ending at index 2, these are all valid:

```
[1,2,1]
[2,1]
[1]
```

That's:

```
right - left + 1
```

subarrays.

This is the key!

So every time the window is valid:

```
count += right - left + 1;
```

10. Example completely

Let's calculate:

```
nums = [1,2,1,3]
k = 2
```

right = 0

Window:

```
[1]
```

Distinct = 1.

Number of valid subarrays ending here:

```
1 - 0 + 1 = 1
```

Count:

```
1
```

right = 1

Window:

```
[1,2]
```

Distinct = 2.

Valid subarrays ending at index 1:

```
[1,2]
[2]
```

Count:

```
2
```

Total:

```
1 + 2 = 3
```

right = 2

Window:

```
[1,2,1]
```

Distinct = 2.

Valid subarrays:

```
[1,2,1]
[2,1]
[1]
```

Count:

```
3
```

Total:

```
6
```

right = 3

Add 3:

```
[1,2,1,3]
```

Distinct = 3.

Too many.

Move left.

```
[2,1,3]
```

Still 3.

Move left.

```
[1,3]
```

Now 2.

So:

```
left = 2
```

Number of valid subarrays ending at index 3:

```
right - left + 1
= 3 - 2 + 1
= 2
```

They are:

```
[1,3]
[3]
```

Final:

```
count = 8
```

Indeed, there are 8 subarrays with at most 2 distinct elements.

11. Now combine this with Binary Search

The answer can only be between:

```
1
```

and

```
number of distinct elements in nums
```

For:

```
nums = [1,2,3]
```

range is:

```
1 ... 3
```

We binary search this range.

For some `mid`, calculate:

```
countAtMost(mid)
```

If:

```
countAtMost(mid) >= target
```

then `mid` is **large enough** to contain the median.

So try smaller:

```
high = mid
```

Otherwise:

```
countAtMost(mid) < target
```

we need a larger value:

```
low = mid + 1
```

12. Complete algorithm

Step 1

Calculate total number of subarrays:

```
long long total = 1LL * n * (n + 1) / 2;
```

Step 2

Calculate the target:

```
long long target = (total + 1) / 2;
```

Step 3

Binary search:

```
low = 1
high = n
```

Step 4

For every `mid`, calculate:

```
number of subarrays having <= mid distinct elements
```

using sliding window.

Step 5

If count is at least target:

```
high = mid
```

otherwise:

```
low = mid + 1
```

At the end:

```
low
```

is the answer.

13. C++ Code

```
class Solution {
public:
    long long countAtMost(vector<int>& nums, int k) {
        int n = nums.size();
        vector<int> freq(100001, 0);

        int left = 0;
        int distinct = 0;
        long long count = 0;

        for (int right = 0; right < n; right++) {
            if (freq[nums[right]] == 0)
                distinct++;
        }
    }
};
```

```

        freq[nums[right]]++;

        while (distinct > k) {
            freq[nums[left]]--;

            if (freq[nums[left]] == 0)
                distinct--;

            left++;
        }

        count += right - left + 1;
    }

    return count;
}

int medianOfUniquenessArray(vector<int>& nums) {
    int n = nums.size();

    long long total = 1LL * n * (n + 1) / 2;
    long long target = (total + 1) / 2;

    int low = 1;
    int high = n;

    while (low < high) {
        int mid = low + (high - low) / 2;

        if (countAtMost(nums, mid) >= target)
            high = mid;
        else
            low = mid + 1;
    }

    return low;
}
};

```

14. Why is the complexity good?

The sliding window takes:

$O(n)$

for each `mid`.

Binary search has:

$O(\log n)$

iterations.

Therefore:

$O(n \log n)$

Overall.

For:

`n <= 100000`

this is easily fast enough.

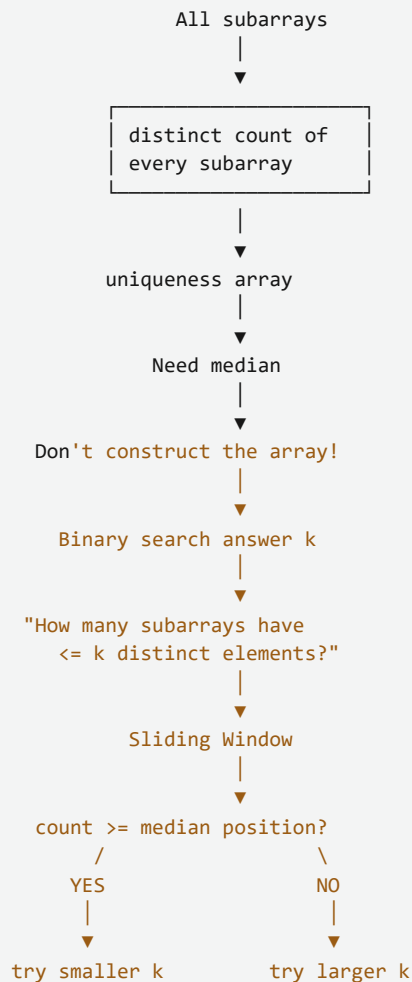
Memory:

$O(\max(\text{nums}[i]))$

because of the frequency array.

15. The entire idea in one picture

Think of the problem like this:



The key transformation is:

Instead of asking:

"What is the median of all distinct counts?"

we ask:

"For a candidate k , how many subarrays have at most k distinct elements?"

That count can be found in $O(n)$ using sliding window, and then **binary search** finds the smallest k that reaches the median position.

If you want, the next step can be to take `nums = [3,4,3,4,5]` and manually execute the **binary search + sliding window on every step**, which makes this problem much easier to understand.

Yes. Let's do a **complete dry run** of `nums = [1,2,3]`, exactly following the code.

```
nums = [1, 2, 3]
n = 3
```

1. First calculate total subarrays

Formula:

```
total = n * (n + 1) / 2
```

So:

```
total = 3 * 4 / 2
      = 6
```

There are 6 subarrays:

```
[1]
[2]
[3]
[1,2]
[2,3]
[1,2,3]
```

Their distinct counts are:

```
[1]      → 1
[2]      → 1
[3]      → 1
[1,2]    → 2
[2,3]    → 2
[1,2,3]  → 3
```

So uniqueness array is:

```
[1, 1, 1, 2, 2, 3]
```

We already know answer = 1, but let's see how the algorithm discovers it.

2. Calculate target

Code:

```
target = (total + 1) / 2;
```

Therefore:

```
target = (6 + 1) / 2
        = 7 / 2
        = 3
```

Why 3?

Because we need the **3rd smallest value**.

```
[1, 1, 1, 2, 2, 3]
      ↑
    3rd
```

So our goal becomes:

```
Find the smallest k such that at least 3 subarrays have <= k distinct elements.
```

3. Start Binary Search

Initially:

```
low = 1
high = n = 3
```

So:

```
low = 1
high = 3
```

Visualize:

```
  1    2    3
  ↑    ↑
low    high
```

4. First Binary Search iteration

Calculate:

```
mid = low + (high - low) / 2
```

So:

```
mid = 1 + (3 - 1) / 2
      = 1 + 1
      = 2
```

We now ask:

How many subarrays have at most 2 distinct elements?

So we call:

```
countAtMost(nums, 2)
```

5. Sliding Window starts

We have:

```
nums = [1, 2, 3]
k = 2
```

Initial state:

```
left = 0
distinct = 0
count = 0
```

Frequency array conceptually:

```
freq[1] = 0
freq[2] = 0
freq[3] = 0
```

right = 0

We add:

```
nums[right] = nums[0] = 1
```

Before adding:

```
freq[1] = 0
```

Therefore this is a new distinct number:

```
distinct++;
```

Now:

```
distinct = 1
freq[1] = 1
```

Window:

```
[1]
  ↑
left
```

We have:

```
distinct = 1
k = 2
```

So the window is valid.

Now:

```
count += right - left + 1;
```

Therefore:

```
count += 0 - 0 + 1
        += 1
```

So:

```
count = 1
```

What subarrays did we count?

```
[1]
```

6. right = 1

Now:

```
nums[right] = nums[1] = 2
```

freq[2] is currently 0, so:

```
distinct = 2  
freq[2] = 1
```

Window:

```
[1, 2]  
  ↑   ↑  
left  right
```

Distinct elements:

```
{1,2}
```

So:

```
distinct = 2  
k = 2
```

Valid.

Calculate:

```
count += right - left + 1
```

```
count += 1 - 0 + 1  
       += 2
```

Therefore:

```
count = 3
```

Which 2 subarrays did we just add?

```
[1,2]  
[2]
```

So far:

```
[1]    → 1  
[1,2]  → 2  
[2]    → 2
```

Total:

```
3
```

7. right = 2

Now add:

```
nums[2] = 3
```

freq[3] = 0, so:

```
distinct = 3
```

Window:

```
[1, 2, 3]
  ↑   ↑
left  right
```

Distinct:

```
{1,2,3}
```

Therefore:

```
distinct = 3
k = 2
```

Oops!

```
3 > 2
```

The window is invalid.

We need to move `left`.

8. Move left

Current:

```
left = 0
right = 2
```

Remove:

```
nums[left] = nums[0] = 1
```

Decrease frequency:

```
freq[1]--
```

So:

```
freq[1] = 0
```

Since its frequency became zero, `1` is no longer in the window.

Therefore:

```
distinct--
```

Now:

```
distinct = 2
left = 1
```

Our window becomes:

```
[2,3]
  ↑  ↑
left right
```

Distinct elements:

```
{2,3}
```

Now:

```
distinct = 2
k = 2
```

Valid!

9. Count the valid subarrays

Now:

```
count += right - left + 1;
```

Substitute:

```
count += 2 - 1 + 1
       += 2
```

So:

```
count = 5
```

What are those two subarrays?

```
[2,3]
[3]
```

So overall we counted:

```
[1]    → 1
[1,2]  → 2
[2]    → 2
[2,3]  → 2
[3]    → 1
```

That's **5 subarrays** with at most 2 distinct elements.

Only this one doesn't qualify:

```
[1,2,3] → 3 distinct
```

Therefore:

```
countAtMost(2) = 5
```

10. Return to Binary Search

Remember:

```
target = 3
```

And we got:

```
count = 5
```

So:

```
5 >= 3
```

Yes!

Therefore `mid = 2` is large enough.

Code executes:

```
high = mid;
```

So:

```
low = 1
high = 2
```

Our search range becomes:

```
1      2
↑      ↑
low    high
```

11. Second Binary Search iteration

Calculate:

```
mid = low + (high - low) / 2
```

```
mid = 1 + (2 - 1) / 2
     = 1 + 0
     = 1
```

Now we ask:

How many subarrays have **at most 1 distinct element**?

Call:

```
countAtMost(nums, 1)
```

12. Sliding Window with k = 1

Initial:

```
left = 0
distinct = 0
count = 0
```

right = 0

Add 1.

Window:

```
[1]
```

Distinct:

```
1
```

Valid because:

```
1 <= k
1 <= 1
```

Count:

```
count += 0 - 0 + 1
       = 1
```

So:

```
count = 1
```

right = 1

Add 2.

Window:

```
[1,2]
```

Distinct:

```
2
```

But:

```
2 > k
2 > 1
```

Invalid.

So move left.

Remove 1.

```
left = 1
distinct = 1
```

Window becomes:

```
[2]
```

Now valid.

Count:

```
count += 1 - 1 + 1
       = 1
```

Total:

```
count = 2
```

The valid subarrays so far:

```
[1]
[2]
```

13. right = 2

Add 3.

Window:

```
[2,3]
```

Distinct:

```
2
```

Again:

```
2 > 1
```

Invalid.

Move left.

Remove 2.

```
left = 2
distinct = 1
```

Window:

```
[3]
```

Valid.

Count:

```
count += 2 - 2 + 1
       = 1
```

Therefore:

```
count = 3
```

So:

```
countAtMost(1) = 3
```

14. Compare with target

We had:

```
target = 3
```

And:

```
count = 3
```

Therefore:

```
count >= target
3 >= 3
```

YES!

So:

```
high = mid;
```

Therefore:

```
low = 1
high = 1
```

15. Binary search ends

Condition:

```
while (low < high)
```

Now:

```
low = 1
high = 1
```

So:

```
low < high
1 < 1
```

False.

Return:

```
return low;
```

Therefore:

```
ANSWER = 1
```

16. Full dry-run table

Here's the whole algorithm in one table:

Binary Search	low	high	mid	countAtMost(mid)	target	Decision
1st	1	3	2	5	3	5 >= 3 → high = 2
2nd	1	2	1	3	3	3 >= 3 → high = 1

Final:

```
low = 1
high = 1

answer = 1
```

17. The most important thing to understand

Don't think that we're actually building:

```
[1,1,1,2,2,3]
```

We never build the uniqueness array.

Instead, binary search asks questions.

Question 1:

```
Is the answer <= 2?
```

We answer this by counting:

```
How many subarrays have <= 2 distinct?
```

Answer:

```
5
```

We need at least:

```
3
```

So yes, answer could be 2 or smaller.

Question 2:

Is the answer ≤ 1 ?

Count:

How many subarrays have ≤ 1 distinct?

Answer:

3

We need:

3

So yes.

Therefore:

answer ≤ 1

Since the minimum possible answer is 1:

answer = 1

One-line intuition

The whole problem boils down to:

Binary search the distinct-count value k , and use sliding window to count how many subarrays have at most k distinct elements. The first k whose count reaches the median position is the answer.

The hardest part is usually understanding why:

count += right - left + 1;

works. Once that is clear, the rest of the solution becomes much easier.